Lab 1. PyTorch and ANNs

This lab is a warm up to get you used to the PyTorch programming environment used in the course, and also to help you review and renew your knowledge of Python and relevant Python libraries. The lab must be done individually. Please recall that the University of Toronto plagarism rules apply.

By the end of this lab, you should be able to:

1. Be able to perform basic PyTorch tensor operations.
2. Be able to load data into PyTorch
3. Be able to configure an Artificial Neural Network (ANN) using PyTorch
4. Be able to train ANNs using PyTorch
5. Be able to evaluate different ANN configuations

You will need to use numpy and PyTorch documentations for this assignment:

- https://docs.scipy.org/doc/numpy/reference/
- https://pytorch.org/docs/stable/torch.html

You can also reference Python API documentations freely.

What to submit

Submit a PDF file containing all your code, outputs, and write-up from parts 1-5. You can produce a PDF of your Google Colab file by going to `File -> Print` and then save as PDF. The Colab instructions has more information.

Do not submit any other files produced by your code.

Include a link to your colab file in your submission.

Please use Google Colab to complete this assignment. If you want to use Jupyter Notebook, please complete the assignment and upload your Jupyter Notebook file to Google Colab for submission.

Adjust the scaling to ensure that the text is not cutoff at the margins.

Colab Link

Submit make sure to include a link to your colab file here

Colab Link: https://drive.google.com/file/d/1FheF7LI-b2ZSfu9BUU1dPTcf6rzv0c64/view?usp=sharing

Part 1. Python Basics [3 pt]

The purpose of this section is to get you used to the basics of Python, including working with functions, numbers, lists, and strings.

Note that we **will** be checking your code for clarity and efficiency.

If you have trouble with this part of the assignment, please review http://cs231n.github.io/python-numpy-tutorial/

Part (a) -- 1pt

Write a function `sum_of_cubes` that computes the sum of cubes up to `n`. If the input to `sum_of_cubes` invalid (e.g. negative or non-integer `n`), the function should print out `"Invalid input"` and return `-1`.

```
def sum_of_cubes(n):
    """Return the sum (1^3 + 2^3 + 3^3 + ... + n^3)

    Precondition: n > 0, type(n) == int

    >>> sum_of_cubes(3)
    36
    >>> sum_of_cubes(1)
    1
    """
    # Check for invalid input
    if type(n) is not int or n <= 0:
        print("Invalid input")
```

```
        print("Invalid input")
        return -1

    # Calculate the sum of cubes
    total = 0
    for i in range(1, n + 1):
        total += i ** 3
    return total

# Test cases
print(sum_of_cubes(3))
print(sum_of_cubes(1))
sum_of_cubes(-1)  # Invalid input
sum_of_cubes(2.5)  # Invalid input
print(sum_of_cubes(5))
sum_of_cubes("three")  # Invalid input
sum_of_cubes("3")  # Invalid input
```

```
36
1
Invalid input
Invalid input
225
Invalid input
Invalid input
-1
```

Part (b) -- 1pt

Write a function `word_lengths` that takes a sentence (string), computes the length of each word in that sentence, and returns the length of each word in a list. You can assume that words are always separated by a space character `" "`.

Hint: recall the `str.split` function in Python. If you arenot sure how this function works, try typing `help(str.split)` into a Python shell, or check out https://docs.python.org/3.6/library/stdtypes.html#str.split

```
help(str.split)
```

Show hidden output

```
def word_lengths(sentence):
    """Return a list containing the length of each word in
    sentence.

    >>> word_lengths("welcome to APS360!")
    [7, 2, 7]
    >>> word_lengths("machine learning is so cool")
    [7, 8, 2, 2, 4]
    """
    # Split the sentence into words
    words = sentence.split()

    # Calculate the length of each word
    lengths = [len(word) for word in words]

    return lengths

# Test cases
print(word_lengths("welcome to APS360!"))
print(word_lengths("machine learning is so cool"))
print(word_lengths("This year is going to be\n great!"))
```

```
[7, 2, 7]
[7, 8, 2, 2, 4]
[4, 4, 2, 5, 2, 2, 6]
```

Part (c) -- 1pt

Write a function `all_same_length` that takes a sentence (string), and checks whether every word in the string is the same length. You should call the function `word_lengths` in the body of this new function.

```
def all_same_length(sentence):
    """Return True if every word in sentence has the same
    length, and False otherwise.
```

```
        >>> all_same_length("all same length")
        False
        >>> all_same_length("hello world")
        True
        """
        # Get the lengths of the words in the sentence
        array = word_lengths(sentence)
        # Check if all lengths are the same
        return all(length == array[0] for length in array)

    # Test cases
    print(all_same_length("all same length"))
    print(all_same_length("hello world"))
    print(all_same_length("test test quiz"))
    print(all_same_length("one two three"))
```

```
False
True
True
False
```

Part 2. NumPy Exercises [5 pt]

In this part of the assignment, you'll be manipulating arrays usign NumPy. Normally, we use the shorter name `np` to represent the package `numpy`.

```
import numpy as np
```

Part (a) -- 1pt

The below variables `matrix` and `vector` are numpy arrays. Explain what you think `<NumpyArray>.size` and `<NumpyArray>.shape` represent.

```
matrix = np.array([[1., 2., 3., 0.5],
                   [4., 5., 0., 0.],
                   [-1., -2., 1., 1.]])
vector = np.array([2., 0., 1., -2.])
```

```
matrix.size
```

```
12
```

```
matrix.shape
```

```
(3, 4)
```

```
vector.size
```

```
4
```

```
vector.shape
```

```
(4,)
```

`<NumpyArray>.size` repersents how many individual elements are in the array, whereas `<NumpyArray>.shape` repersents the dimensions of the array (how many rows and columns are in the array).

Part (b) -- 1pt

Perform matrix multiplication `output = matrix x vector` by using for loops to iterate through the columns and rows. Do not use any builtin NumPy functions. Cast your output into a NumPy array, if it isn't one already.

Hint: be mindful of the dimension of output

```
output = None
```

```
# Initialize the output vector with zeros
output = [0.0] * matrix.shape[0]

# matrix multiplication using for loops
for i in range(matrix.shape[0]):
    for j in range(vector.shape[0]):
        output[i] += matrix[i, j] * vector[j]

print(output)
```

```
[np.float64(4.0), np.float64(8.0), np.float64(-3.0)]
```

Part (c) -- 1pt

Perform matrix multiplication `output2 = matrix x vector` by using the function `numpy.dot`.

We will never actually write code as in part(c), not only because `numpy.dot` is more concise and easier to read/write, but also performance-wise `numpy.dot` is much faster (it is written in C and highly optimized). In general, we will avoid for loops in our code.

```
output2 = None
```

```
output2 = np.dot(matrix, vector)
print(output2)
```

```
[ 4.  8. -3.]
```

Part (d) -- 1pt

As a way to test for consistency, show that the two outputs match.

```
if np.array_equal(output, output2):
    print("The outputs are the same.")
else:
    print("The outputs are different.")
```

```
The outputs are the same.
```

Part (e) -- 1pt

Show that using `np.dot` is faster than using your code from part (c).

You may find the below code snippit helpful:

```
import time

# record the time before running code
start_time = time.time()

# place code to run here
for i in range(10000):
    99*99

# record the time after the code is run
end_time = time.time()

# compute the difference
diff = end_time - start_time
diff
```

Show hidden output

```
import time

# Measure the time taken to run the code using the for loop

# Initialize the matrix and vetor to be large so that the time taken to run the code is significant
matrix = np.random.rand(100, 100)
vector = np.random.rand(100)
```

```
#Initialize the output vector with zeros
output = [0.0] * matrix.shape[0]

start_time_for = time.time()

for i in range(matrix.shape[0]):
    for j in range(vector.shape[0]):
        output[i] += matrix[i, j] * vector[j]

end_time_for = time.time()
print("Time taken by for loops: {:.10f} seconds".format(
    end_time_for - start_time_for
))

# Measure the time taken to run the code using numpy's dot function
start_time_dot = time.time()

output_dot = np.dot(matrix, vector)

end_time_dot = time.time()
print("Time taken by numpy's dot function: {:.10f} seconds".format(
    end_time_dot - start_time_dot
))

#confirm that the outputs are the same
if np.allclose(output, output_dot):
    print("The outputs are the same.")
else:
    print("The outputs are different.")
```

```
Time taken by for loops: 0.0019974709 seconds
Time taken by numpy's dot function: 0.0000000000 seconds
The outputs are the same.
```

Part 3. Images [6 pt]

A picture or image can be represented as a NumPy array of "pixels", with dimensions H × W × C, where H is the height of the image, W is the width of the image, and C is the number of colour channels. Typically we will use an image with channels that give the the Red, Green, and Blue "level" of each pixel, which is referred to with the short form RGB.

You will write Python code to load an image, and perform several array manipulations to the image and visualize their effects.

```
import matplotlib.pyplot as plt
```

Part (a) -- 1 pt

This is a photograph of a dog whose name is Mochi.



Load the image from its url (https://drive.google.com/uc?export=view&id=1oaLVR2hr1_qzpKQ47i9rVUIklwbDcews) into the variable `img` using the `plt.imread` function.

Hint: You can enter the URL directly into the `plt.imread` function as a Python string.

```
img = None
url = "https://drive.google.com/uc?export=view&id=1oaLVR2hr1_qzpKQ47i9rVUIklwbDcews"

import urllib.request
img = plt.imread(urllib.request.urlretrieve(url)[0])
```

Part (b) -- 1pt

Use the function `plt.imshow` to visualize `img`.

This function will also show the coordinate system used to identify pixels. The origin is at the top left corner, and the first dimension indicates the Y (row) direction, and the second dimension indicates the X (column) dimension.

```
plt.imshow(img)
```

```
<matplotlib.image.AxesImage at 0x15e7455e7d0>
```



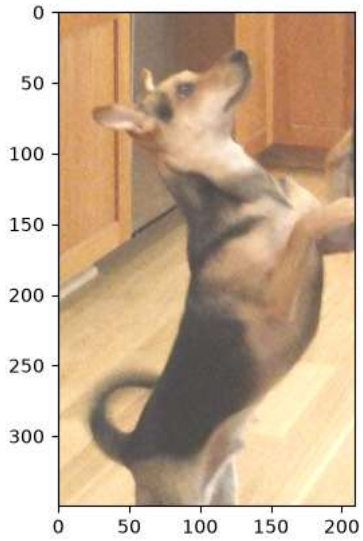
Part (c) -- 2pt

Modify the image by adding a constant value of 0.25 to each pixel in the `img` and store the result in the variable `img_add`. Note that, since the range for the pixels needs to be between [0, 1], you will also need to clip img_add to be in the range [0, 1] using `numpy.clip`. Clipping sets any value that is outside of the desired range to the closest endpoint. Display the image using `plt.imshow`.

```
img_add = None
import numpy
#adding 0.25 to eacg pixel in the image
img_add = img + 0.25
numpy.clip(img_add, 0, 1, out=img_add)  # Ensure pixel values are in [0, 1]

#show the image after adding 0.25 to each pixel
plt.imshow(img_add)
```

```
<matplotlib.image.AxesImage at 0x15e7459cb90>
```



Part (d) -- 2pt

Crop the **original** image (`img` variable) to a 130 x 150 image including Mochi's face. Discard the alpha colour channel (i.e. resulting `img_cropped` should **only have RGB channels**)

Display the image.

```
img_crop = None

#cropping the image to only show mochi's face
img_crop = img[0:130, 10:160]

#discard alpha colour channel
img_crop = img_crop[:, :, :3]

plt.imshow(img_crop)
```

```
<matplotlib.image.AxesImage at 0x15e692ae090>
```



Part 4. Basics of PyTorch [6 pt]

PyTorch is a Python-based neural networks package. Along with tensorflow, PyTorch is currently one of the most popular machine learning libraries.

PyTorch, at its core, is similar to Numpy in a sense that they both try to make it easier to write codes for scientific computing achieve improved performance over vanilla Python by leveraging highly optimized C back-end. However, compare to Numpy, PyTorch offers much

better GPU support and provides many high-level features for machine learning. Technically, Numpy can be used to perform almost every thing PyTorch does. However, Numpy would be a lot slower than PyTorch, especially with CUDA GPU, and it would take more effort to write machine learning related code compared to using PyTorch.

```
import torch
```

Part (a) -- 1 pt

Use the function `torch.from_numpy` to convert the numpy array `img_cropped` into a PyTorch tensor. Save the result in a variable called `img_torch`.

```
img_torch = None
img_torch = torch.from_numpy(img_crop).float()
```

Part (b) -- 1pt

Use the method `<Tensor>.shape` to find the shape (dimension and size) of `img_torch`.

```
img_torch.shape
# torch.Size([130, 150, 3])
```

```
torch.Size([130, 150, 3])
```

Part (c) -- 1pt

How many floating-point numbers are stored in the tensor `img_torch`?

```
img_torch.numel()
# 58500 fp numbers stored in the tensor img_torch
# from the torch size of 130*150*3=58500
```

```
58500
```

Part (d) -- 1 pt

What does the code `img_torch.transpose(0,2)` do? What does the expression return? Is the original variable `img_torch` updated? Explain.

```
print(img_torch.transpose(0,2).shape)
# img_torch.transpose(0,2) swaps dimensions 0 and 2 of the tensor, changing the shape from [130, 150, 3] to [3, 150, 130]. This
img_torch.shape
# the original variable does not get updated, so img_torch still has the shape [130, 150, 3] after the transpose operation. To
```

```
torch.Size([3, 150, 130])
torch.Size([130, 150, 3])
```

Part (e) -- 1 pt

What does the code `img_torch.unsqueeze(0)` do? What does the expression return? Is the original variable `img_torch` updated? Explain.

```
print(img_torch.unsqueeze(0).shape)
# img_torch.unsqueeze(0) adds a new dimension at index 0 of the tensor, changing the shape from [130, 150, 3] to [1, 130, 150,
img_torch.shape
# the original variable does not get updated, so img_torch still has the shape [130, 150, 3] after the unsqueeze operation. To
```

```
torch.Size([1, 130, 150, 3])
torch.Size([130, 150, 3])
```

Part (f) -- 1 pt

Find the maximum value of `img_torch` along each colour channel? Your output should be a one-dimensional PyTorch tensor with exactly three values.

Hint: lookup the function `torch.max`.

```
#/help(torch.max)

# find maximum value in the red channel of the image
channel_red = img_torch[:, :, 0]  # Extract the red channel
channel_red.max()
print("The maximum value in the red channel is:", channel_red.max().item())

# find maximum value in the green channel of the image
channel_green = img_torch[:, :, 1]  # Extract the green channel
channel_green.max()
print("The maximum value in the green channel is:", channel_green.max().item())

#find maximum value in the blue channel of the image
channel_blue = img_torch[:, :, 2]  # Extract the blue channel
print("The maximum value in the blue channel is:", channel_blue.max().item())

# now make the output tensor with all the maximum values in each channel
max_values = torch.tensor([channel_red.max(), channel_green.max(), channel_blue.max()])
print("The maximum values in each channel are:", max_values)
```

```
The maximum value in the red channel is: 0.8941176533699036
The maximum value in the green channel is: 0.7882353067398071
The maximum value in the blue channel is: 0.6745098233222961
The maximum values in each channel are: tensor([0.8941, 0.7882, 0.6745])
```

Part 5. Training an ANN [10 pt]

The sample code provided below is a 2-layer ANN trained on the MNIST dataset to identify digits less than 3 or greater than and equal to 3. Modify the code by changing any of the following and observe how the accuracy and error are affected:

- number of training iterations
- number of hidden units
- numbers of layers
- types of activation functions
- learning rate

Please select at least three different options from the list above. For each option, please select two to three different parameters and provide a table.

```
import torch
import torch.nn as nn
import torch.nn.functional as F
from torchvision import datasets, transforms
import matplotlib.pyplot as plt # for plotting
import torch.optim as optim

torch.manual_seed(1) # set the random seed

# define a 2-layer artificial neural network
class Pigeon(nn.Module):
    def __init__(self):
        super(Pigeon, self).__init__()
        self.layer1 = nn.Linear(28 * 28, 30)
        self.layer2 = nn.Linear(30, 1)
    def forward(self, img):
        flattened = img.view(-1, 28 * 28)
        activation1 = self.layer1(flattened)
        activation1 = F.relu(activation1)
        activation2 = self.layer2(activation1)
        return activation2

pigeon = Pigeon()

# load the data
mnist_data = datasets.MNIST('data', train=True, download=True)
mnist_data = list(mnist_data)
mnist_train = mnist_data[:1000]
mnist_val   = mnist_data[1000:2000]
```

```
mnist_val = mnist_data[1000:2000]
img_to_tensor = transforms.ToTensor()


# simplified training code to train `pigeon` on the "small digit recognition" task
criterion = nn.BCEWithLogitsLoss()
optimizer = optim.SGD(pigeon.parameters(), lr=0.005, momentum=0.9)

# number of iterations
num_iterations = 1

# training loop
for i in range(num_iterations):
    for (image, label) in mnist_train:
        # actual ground truth: is the digit less than 3?
        actual = torch.tensor(label < 3).reshape([1,1]).type(torch.FloatTensor)
        # pigeon prediction
        out = pigeon(img_to_tensor(image)) # step 1-2
        # update the parameters based on the loss
        loss = criterion(out, actual)      # step 3
        loss.backward()                    # step 4 (compute the updates for each parameter)
        optimizer.step()                   # step 4 (make the updates for each parameter)
        optimizer.zero_grad()              # a clean up step for PyTorch

# computing the error and accuracy on the training set
error = 0
for (image, label) in mnist_train:
    prob = torch.sigmoid(pigeon(img_to_tensor(image)))
    if (prob < 0.5 and label < 3) or (prob >= 0.5 and label >= 3):
        error += 1
print("Training Error Rate:", error/len(mnist_train))
print("Training Accuracy:", 1 - error/len(mnist_train))


# computing the error and accuracy on a test set
error = 0
for (image, label) in mnist_val:
    prob = torch.sigmoid(pigeon(img_to_tensor(image)))
    if (prob < 0.5 and label < 3) or (prob >= 0.5 and label >= 3):
        error += 1
print("Test Error Rate:", error/len(mnist_val))
print("Test Accuracy:", 1 - error/len(mnist_val))
```

Show hidden output

Part 5 Experiment Summary

The baseline model uses 1,000 training images, 30 hidden units, ReLU activation, a learning rate of `0.005`, and one training pass through the data.

1. Training Iterations

Training iterations	Training accuracy	Testing accuracy
1	96.4%	92.1%
2	98.4%	94.3%
3	98.6%	93.1%

Additional iterations give the optimizer more opportunities to update the weights using gradient descent, reducing the training loss and improving training accuracy. However, after two iterations, the model begins fitting the training examples too closely. This mild overfitting explains why training accuracy continued to increase while testing accuracy decreased slightly after three iterations.

2. Number of Hidden Units

Hidden units	Training accuracy	Testing accuracy
15	96.2%	91.3%
30	96.4%	92.1%
60	96.9%	92.0%

Increasing the number of hidden units increases the model's capacity because it provides more learnable weights and allows the network to represent more complex patterns. This slightly improved training accuracy. However, the testing accuracy changed very little, suggesting that the task and limited training set did not require the additional capacity. Extra units can also increase the risk of learning noise rather than generalizable features.

3. Number of Layers

Network architecture	Training accuracy	Testing accuracy
One hidden layer	96.4%	92.1%
Two hidden layers	96.1%	91.3%
Three hidden layers	95.9%	90.7%

Deeper networks can learn hierarchical representations, where earlier layers detect simpler patterns and later layers combine them into more complex features. However, adding layers also introduces more parameters and makes optimization more difficult. During backpropagation, gradients must pass through each layer, so they can become very small or unstable. With only 1,000 training examples, the deeper networks were harder to optimize and produced lower accuracy.

4. Activation Function

Activation function	Training accuracy	Testing accuracy
ReLU	96.4%	92.1%
Sigmoid	92.7%	88.3%
Tanh	96.0%	90.6%

ReLU outputs zero for negative inputs and keeps positive inputs unchanged. This produces useful gradients for active neurons and generally makes gradient-based learning easier.

Sigmoid and Tanh compress their inputs into limited ranges. For large positive or negative inputs, their gradients become very small, causing the vanishing-gradient problem. As a result, earlier layers receive weaker updates and learn more slowly. ReLU therefore achieved the best performance in this experiment.

5. Learning Rate

Learning rate	Training accuracy	Testing accuracy
0.001	92.2%	88.7%
0.005	96.4%	92.1%
0.01	96.1%	91.8%

The learning rate controls the size of each gradient-descent update. With a learning rate of `0.001`, the updates were very small, so the model learned slowly and did not reach as effective a set of weights after one training pass.

The rate of `0.005` provided a good balance between learning speed and stability. Increasing the rate to `0.01` made the updates more drastic. These larger updates can overshoot low points in the loss function instead of settling near them, which explains the slight decrease in training and testing accuracy.

Part (a) -- 3 pt

Comment on which of the above changes resulted in the best accuracy on training data? What accuracy were you able to achieve?

The highest training accuracy was achieved by using **3 training iterations**, with an accuracy of **98.6%**. This was higher than the results from changing the hidden units, number of layers, activation function, or learning rate. Increasing the iterations allowed the model to perform more weight updates and fit the training data more effectively.

Part (b) -- 3 pt

Comment on which of the above changes resulted in the best accuracy on testing data? What accuracy were you able to achieve?

The best testing accuracy was achieved with **2 training iterations**, reaching **94.3%**. This was higher than the baseline accuracy of 92.1% and the 3-iteration result of 93.1%. Additional training improved performance initially, but three iterations caused slight overfitting.

Part (c) -- 4 pt

Which model hyperparameters should you use, the ones from (a) or (b)?

We should use the hyperparameters from **(b)** because they produced the highest testing accuracy and generalized better to unseen data. Although the hyperparameters from **(a)** achieved higher training accuracy, this may indicate overfitting. An overfit model memorizes the training data rather than learning patterns that generalize well, which can lead to less accurate predictions on new data.